

# Teorie si cod pentru colocviul de IA

Fisa de invatat: NumPy, Naive Bayes, KNN, normalizare, SVM, kernel-uri, KRR, Markov features, perceptron, MLP si features de imagine

Scopul documentului: sa ai intr-un singur PDF teoria practica si template-urile de cod pe care le poti adapta rapid la variantele de colocviu. Este scris pentru invatare rapida, nu pentru demonstratii matematice lungi.

# Cuprins rapid

- 1. Pipeline-ul general pentru orice subiect
- 2. NumPy: array-uri, shape, axis, reshape, ravel, load, savetxt
- 3. Train / validation split, accuracy si raport
- 4. Naive Bayes si discretizare
- 5. KNN: L1, L2, Hamming, similaritate string kernel
- 6. Normalizare, Bag-of-Words si SVM
- 7. Kernel-uri: string kernel, intersection kernel, matrice kernel
- 8. KRR: Kernel Ridge Regression, np.rint, alpha
- 9. Semnale accelerometru: padding, flatten, Markov features
- 10. Perceptron, Widrow-Hoff, MLPClassifier si PyTorch minim
- 11. Laborator 7: features de imagine, gradient, Hamming, SVM intersection
- 12. Template-uri finale de cod si greseli frecvente

## 1. Pipeline-ul general pentru orice subiect

In aproape orice varianta de colocviu ai aceeasi ordine de lucru. Daca o memorezi, nu te pierzi in cerinta.

| Pas | Ce faci                                            | De ce                                                         |
|-----|----------------------------------------------------|---------------------------------------------------------------|
| 1   | Citesti datele: .npy, .txt, .csv sau imagini       | Trebuie sa obtii X_train, y_train, X_test                     |
| 2   | Transformi datele in vectori de aceeasi dimensiune | Modelele ML cer matrice X de forma nr_exemple x nr_features   |
| 3   | Faci train/validation split                        | Nu ai etichetele de test, deci validezi pe o bucata din train |
| 4   | Testezi mai multi hiperparametri                   | Alegi modelul cu accuracy cea mai buna pe validation          |
| 5   | Antrenezi pe train complet                         | Folosesti toate datele etichetate inainte de submitie         |
| 6   | Prezici pe test si salvezi .txt                    | Submisia cere cate o predictie pe linie                       |
| 7   | Scrii raport                                       | Tabel cu model, hiperparametri si validation accuracy         |

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# X = features, y = etichete
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# antrenezi modelul
model.fit(X_train, y_train)

# validezi
pred_val = model.predict(X_val)
acc = accuracy_score(y_val, pred_val)
print("validation accuracy:", acc)

# submitie
pred_test = model.predict(X_test)
```

```
np.savetxt("predictii.txt", pred_test, fmt="%d")
```

Regula mare: X\_train trebuie sa fie matrice 2D: (numar\_exemple, numar\_features). Daca exemplele au lungimi diferite, faci padding sau truncare.

## 2. NumPy - baza pentru toate laboratoarele

NumPy este biblioteca principala pentru vectori, matrici, imagini, features si predictii. In subiecte vei folosi permanent np.array, shape, slicing, reshape, ravel, sum, mean, std, argsort, bincount si savetxt.

### 2.1 Array, shape si dimensiuni

```
import numpy as np

x = np.array([1, 2, 3])
print(x.shape)      # (3,)

X = np.array([[1, 2, 3], [4, 5, 6]])
print(X.shape)      # (2, 3) = 2 linii, 3 coloane
```

In ML, forma standard este:

```
X.shape = (numar_exemple, numar_features)
y.shape = (numar_exemple,)
```

### 2.2 Liste cu lungimi diferite

O matrice normala trebuie sa aiba acelasi numar de elemente pe fiecare linie.

```
# Gresit pentru ML: prima lista are 2 elemente, a doua are 3
X = [[1, 2], [3, 4, 5]]

# Corect prin padding:
X = np.array([[1, 2, 0], [3, 4, 5]])

# Corect prin truncare:
X = np.array([[1, 2], [3, 4]])
```

Padding = adaug zerouri la exemplele scurte. Truncare = tai exemplele lungi. Asta apare direct la semnalele de accelerometru.

### 2.3 np.load, np.loadtxt, np.savetxt

```
# Pentru fisiere .npy cu numere
X = np.load("train_data.npy")

# Pentru .npy cu string-uri/text
texts = np.load("train_data.npy", allow_pickle=True)

# Pentru fisiere .txt numerice
X = np.loadtxt("train_images.txt")
y = np.loadtxt("train_labels.txt", dtype=int)

# Salvare predictii, cate una pe linie
np.savetxt("predictii.txt", pred_test, fmt="%d")
```

np.savetxt cu fmt='%d' salveaza etichete intregi. Este necesar la submisii.

### 2.4 reshape si ravel

reshape schimba forma, ravel transforma in vector 1D.

```
image_vector = np.arange(784)
image = np.reshape(image_vector, (28, 28)) # pentru afisare MNIST

A = np.array([[1, 2], [3, 4]])
v = np.ravel(A)                        # [1, 2, 3, 4]

# Pentru semnal accelerometru lungime 100, 3 axe:
signal = np.zeros((100, 3))
signal_vector = signal.reshape(-1)    # 300 valori
```

## 2.5 axis - cand folosesti fiecare tip

axis spune pe ce directie aplici operatia. Pentru operatii ca mean/sum/std, axa indicata dispare din rezultat.

```
X = np.array([[1, 2, 3],
              [4, 5, 6]])

np.sum(X)          # 21, suma tuturor valorilor
np.sum(X, axis=0)  # [5, 7, 9], suma pe coloane
np.sum(X, axis=1)  # [6, 15], suma pe linii
```

| Date                          | Cod                        | Rezultat       | Cand folosesc                         |
|-------------------------------|----------------------------|----------------|---------------------------------------|
| X = (nr_exemple, nr_features) | np.mean(X, axis=0)         | (nr_features,) | media fiecarui feature, standardizare |
| X = (nr_exemple, nr_features) | np.mean(X, axis=1)         | (nr_exemple,)  | media fiecarui exemplu                |
| images = (nr_img, h, w)       | np.mean(images, axis=0)    | (h, w)         | imaginea medie                        |
| images = (nr_img, h, w)       | np.sum(images, axis=(1,2)) | (nr_img,)      | suma pixelilor pentru fiecare imagine |
| signal = (timp, 3)            | np.mean(signal, axis=0)    | (3,)           | media pe axele x, y, z                |

## 2.6 argsort si bincount pentru KNN

```
distances = np.array([2.5, 0.3, 1.2, 5.0])
idx = np.argsort(distances)[:3] # indicii celor mai mici 3 distante

similarities = np.array([5, 0, 12, 2])
idx = np.argsort(-similarities)[:3] # indicii celor mai mari 3 scoruri

labels = np.array([1, 1, 2])
pred = np.bincount(labels).argmax() # clasa majoritara = 1
```

## 3. Train / validation split, accuracy si raport

Pentru ca testul nu are etichete, folosesti o parte din train ca validare. Acolo alegi k, alpha, C, learning rate etc.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_tr, X_val, y_tr, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# random_state=42 fixeaza aleatorul, ca sa obtii acelasi split la fiecare rulare.
# stratify=y pastreaza proportia claselor in train si validation.
```

Accuracy este proportia exemplelor clasificate corect:

```
acc = accuracy_score(y_val, pred_val)
# echivalent cu:
acc_manual = np.mean(pred_val == y_val)
```

In raport pui un tabel simplu:

| Model | Features                  | Hiperparametri            | Validation accuracy |
|-------|---------------------------|---------------------------|---------------------|
| KNN   | string kernel p=8         | k=3                       | ...                 |
| KRR   | precomputed string kernel | alpha=1e-3                | ...                 |
| SVM   | Markov features           | C=10, intersection kernel | ...                 |

## 4. Naive Bayes si discretizare

Naive Bayes apare pe MNIST: fiecare pixel este tratat ca un atribut, iar valorile continue ale pixelilor se discretizeaza in intervale.

Ideea: pentru fiecare clasa  $c$  calculezi  $\log P(c) + \text{suma } \log P(x_i | c)$ . Alegi clasa cu scorul maxim.

### 4.1 Discretizare cu bins

```
num_bins = 5
bins = np.linspace(start=0, stop=255, num=num_bins)
X_binned = np.digitize(X, bins)

# np.digitize transforma fiecare pixel in indexul intervalului in care cade.
```

### 4.2 Formula

Pentru un exemplu  $X = [x_1, x_2, \dots, x_n]$ :

```
score(c) = log(P(c)) + sum_i log(P(x_i | c))
predictie = clasa cu score(c) maxim
```

### 4.3 Template simplificat

```
class NaiveBayesClassifier:
    def __init__(self, num_classes, num_bins):
        self.num_classes = num_classes
        self.num_bins = num_bins

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.class_log_prior = np.zeros(self.num_classes)
        self.feature_log_prob = np.zeros((self.num_classes, n_features, self.num_bins + 1))

        for c in range(self.num_classes):
            X_c = X[y == c]
            self.class_log_prior[c] = np.log(len(X_c) / n_samples)

            for j in range(n_features):
                counts = np.bincount(X_c[:, j], minlength=self.num_bins + 1) + 1
                probs = counts / counts.sum()
                self.feature_log_prob[c, j] = np.log(probs)

    def predict(self, X):
        preds = []
        for x in X:
            scores = []
            for c in range(self.num_classes):
                score = self.class_log_prior[c]
                for j, value in enumerate(x):
                    score += self.feature_log_prob[c, j, value]
                scores.append(score)
            preds.append(np.argmax(scores))
        return np.array(preds)
```

+1 in counts este Laplace smoothing. Evita probabilitati 0, iar  $\log(0)$  ar fi imposibil.

## 5. KNN - metoda celor mai apropiati vecini

KNN nu antreneaza coeficienti. El memoreaza train-ul. Pentru un exemplu nou, calculeaza distante/similaritati fata de toate exemplele de train, ia cei mai apropiati k vecini si face vot majoritar.

### 5.1 Distanțe importante

```
def l1_distance(x, y):
    return np.sum(np.abs(x - y))

# L2 / Euclidean
def l2_distance(x, y):
    return np.sqrt(np.sum((x - y) ** 2))
```

```
# Hamming pentru vectori binari/categorici
def hamming_distance(x, y):
    return np.sum(x != y)
```

## 5.2 KNN numeric implementat manual

```
def knn_predict_one(X_train, y_train, x_test, k=3, metric="l2"):
    distances = []
    for x in X_train:
        if metric == "l1":
            d = np.sum(np.abs(x_test - x))
        else:
            d = np.sqrt(np.sum((x_test - x) ** 2))
        distances.append(d)

    distances = np.array(distances)
    idx = np.argsort(distances)[:k]
    neighbor_labels = y_train[idx]
    return np.bincount(neighbor_labels).argmax()

pred = np.array([knn_predict_one(X_train, y_train, x, k=3, metric="l1") for x in X_val])
```

## 5.3 KNN cu sklearn

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=3, metric="manhattan")
model.fit(X_train, y_train)
pred = model.predict(X_val)
```

manhattan = L1. euclidean = L2.

# 6. Normalizare, Bag-of-Words si SVM

## 6.1 Standardizare

Standardizarea transforma fiecare feature astfel incat sa aiba medie 0 si deviatie standard 1. Se face fit doar pe train, apoi transform pe train/val/test.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)

X_train_scaled = scaler.transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

## 6.2 Normalizare L1/L2

```
from sklearn.preprocessing import Normalizer

normalizer = Normalizer(norm="l2") # sau norm="l1"
X_train_norm = normalizer.transform(X_train)
X_val_norm = normalizer.transform(X_val)
```

StandardScaler normalizeaza pe coloane/features. Normalizer L1/L2 normalizeaza fiecare exemplu/vector separat.

## 6.3 Bag-of-Words

Bag-of-Words transforma texte in vectori: fiecare pozitie corespunde unui cuvant din vocabular, iar valoarea este numarul de aparitii.

```
def build_vocab(texts):
    vocab = {}
    for text in texts:
        for word in text.split():
            if word not in vocab:
                vocab[word] = len(vocab)
    return vocab

def bow_transform(texts, vocab):
```

```

X = np.zeros((len(texts), len(vocab)))
for i, text in enumerate(texts):
    for word in text.split():
        if word in vocab:
            X[i, vocab[word]] += 1
return X

vocab = build_vocab(train_texts)
X_train_bow = bow_transform(train_texts, vocab)
X_test_bow = bow_transform(test_texts, vocab)

```

## 6.4 SVM

```

from sklearn.svm import SVC

model = SVC(C=1, kernel="linear")
model.fit(X_train, y_train)
pred = model.predict(X_val)

# Variante utile:
# SVC(C=1, kernel="rbf", gamma="scale")
# SVC(C=10, kernel="precomputed")

```

C controleaza penalizarea greselilor. C mare = model mai strict pe train, risc de overfitting. C mic = model mai regularizat.

## 7. Kernel-uri si matrice kernel

Un kernel este o functie de similaritate: primeste doua exemple si returneaza un numar. Matricea kernel aplica functia pentru toate perechile.

$K_{train}[i, j] = \text{kernel}(X_{train}[i], X_{train}[j])$ ;  $K_{test}[i, j] = \text{kernel}(X_{test}[i], X_{train}[j])$ . La predictie, testul se compara cu train-ul, nu cu el insusi.

### 7.1 Template general

```

def build_kernel_matrix(X1, X2, kernel_function):
    K = np.zeros((len(X1), len(X2)))
    for i in range(len(X1)):
        for j in range(len(X2)):
            K[i, j] = kernel_function(X1[i], X2[j])
    return K

K_train = build_kernel_matrix(X_train, X_train, kernel_function)
K_val = build_kernel_matrix(X_val, X_train, kernel_function)
K_test = build_kernel_matrix(X_test, X_train, kernel_function)

```

### 7.2 String kernel pe n-grame

Pentru texte, subiectul cere n-grame de caractere de lungime  $p=8$ . Similaritatea este numarul de n-grame comune. Cu set, conteaza prezenta, nu frecventa.

```

def get_ngrams(text, p=8):
    return set(text[i:i+p] for i in range(len(text) - p + 1))

def string_kernel(s, t, p=8):
    ngrams_s = get_ngrams(s, p)
    ngrams_t = get_ngrams(t, p)
    return len(ngrams_s.intersection(ngrams_t))

# Exemplu cu p=4:
# "ananas" -> {"anan", "nana", "anas"}
# "banana" -> {"bana", "anan", "nana"}
# intersectie = {"anan", "nana"}, scor = 2

```

### 7.3 KNN cu string kernel

```

def predict_knn_string(train_texts, train_labels, test_text, k=3, p=8):
    sims = []
    for train_text in train_texts:
        sims.append(string_kernel(test_text, train_text, p=p))
    sims = np.array(sims)

```

```

idx = np.argsort(-sims)[:k]    # cele mai mari similaritati
labels = train_labels[idx]
return np.bincount(labels).argmax()

pred = np.array([
    predict_knn_string(train_texts, train_labels, text, k=3, p=8)
    for text in test_texts
])

```

Pasii: ia un text de test, il compara cu toate textele de train, salveaza scorurile, sorteaza descrescator, ia primii k vecini, apoi vot majoritar pe etichetele lor.

## 7.4 Kernel intersecție

Kernel-ul intersecție se foloseste pe histograme sau features pozitive, de exemplu histograme din imagini sau Markov features.

```

def intersection_kernel_vector(x, y):
    return np.sum(np.minimum(x, y))

# exemplu: x=[1,4,2], y=[3,2,5]
# sum(min) = 1 + 2 + 2 = 5

def intersection_kernel_matrix(X1, X2):
    K = np.zeros((X1.shape[0], X2.shape[0]))
    for i in range(X1.shape[0]):
        for j in range(X2.shape[0]):
            K[i, j] = np.sum(np.minimum(X1[i], X2[j]))
    return K

```

## 7.5 SVM cu kernel precomputed

```

from sklearn.svm import SVC

K_train = intersection_kernel_matrix(X_train, X_train)
K_val = intersection_kernel_matrix(X_val, X_train)

model = SVC(C=10, kernel="precomputed")
model.fit(K_train, y_train)
pred_val = model.predict(K_val)

```

Gresit: `model.fit(X_train, y_train)` dupa ce ai pus `kernel='precomputed'`. Corect: `model.fit(K_train, y_train)`.

# 8. KRR - Kernel Ridge Regression

KRR foloseste kernel-uri ca sa prezica un scor numeric. In colocviu il folosesti pentru clasificare: prezice scoruri reale, apoi le rotunjesti la clase intregi.

## 8.1 Cum functioneaza

KRR invata coeficienti pentru exemplele de train. Pentru un exemplu nou x, predictia este o combinatie a similaritatilor dintre x si fiecare exemplu de train.

```

f(x) = coef_1 * K(x, train_1)
      + coef_2 * K(x, train_2)
      + ...
      + coef_n * K(x, train_n)

```

## 8.2 Cod KRR cu kernel precomputed

```

from sklearn.kernel_ridge import KernelRidge
from sklearn.metrics import accuracy_score

model = KernelRidge(alpha=0.001, kernel="precomputed")
model.fit(K_train, y_train)

pred_scores = model.predict(K_val)
pred = np rint(pred_scores).astype(int)
pred = np.clip(pred, y_train.min(), y_train.max())

acc = accuracy_score(y_val, pred)
print(acc)

```



## 8.3 Ce face np rint

np rint rotunjeste la cel mai apropiat intreg, dar rezultatul ramane float pana faci astype(int).

```
pred_scores = np.array([0.02, 1.13, 2.89, 2.05, 0.77])
np rint(pred_scores) # [0., 1., 3., 2., 1.]
np rint(pred_scores).astype(int) # [0, 1, 3, 2, 1]
```

## 8.4 De ce folosim np clip

KRR poate prezice scoruri in afara claselor valide. Daca clasele sunt 0,1,2,3, un scor 3.7 ar deveni 4 dupa rotunjire, ceea ce este invalid.

```
pred = np rint(pred_scores).astype(int)
pred = np clip(pred, y_train.min(), y_train.max())
```

## 8.5 Ce face alpha

alpha este regularizarea. Controleaza cat de mult se potriveste modelul pe train.

| Valoare alpha      | Efect                                       | Risc            |
|--------------------|---------------------------------------------|-----------------|
| foarte mic: 1e-5   | model flexibil, se potriveste tare pe train | overfitting     |
| mediu: 1e-3 / 1e-2 | echilibru posibil                           | depinde de date |
| mare: 1e-1 / 1     | model mai rigid, coeficienti mai mici       | underfitting    |

```
alphas = [1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1]

best_alpha = None
best_acc = 0
for alpha in alphas:
    model = KernelRidge(alpha=alpha, kernel="precomputed")
    model.fit(K_train, y_train)

    pred_scores = model.predict(K_val)
    pred = np rint(pred_scores).astype(int)
    pred = np clip(pred, y_train.min(), y_train.max())

    acc = accuracy_score(y_val, pred)
    print(alpha, acc)
    if acc > best_acc:
        best_alpha = alpha
        best_acc = acc
```

# 9. Semnale accelerometru si Markov features

In varianta cu accelerometru, fiecare fisier .txt contine un semnal cu 3 coloane: x, y, z. Lungimile difera, deci trebuie normalizate prin taiere sau padding.

## 9.1 Citire si fixare lungime

```
def fix_length(signal, target_len):
    if signal.shape[0] > target_len:
        return signal[:target_len]
    else:
        pad_len = target_len - signal.shape[0]
        return np.pad(signal, ((0, pad_len), (0, 0)), mode="constant")

signal = np.loadtxt("data/train/example.txt") # shape = (lungime, 3)
signal_fixed = fix_length(signal, target_len=150)
vector = signal_fixed.reshape(-1) # pentru MLP
```

## 9.2 Markov features - ideea

Pentru fiecare axa x/y/z: discretizezi valorile in k intervale, inlocuiesti valorile cu indexul intervalului, numeri trecerile i -> j si normalizezi liniile matricei. Apoi liniarizezi cele 3 matrici si le concatenezi.

Daca  $k=6$ , o matrice are  $6 \times 6 = 36$  valori. Pentru 3 axe ai  $3 \times 36 = 108$  features.

## 9.3 Cod Markov features

```
def markov_axis_features(values, k=6, min_val=None, max_val=None):
    if min_val is None:
        min_val = values.min()
    if max_val is None:
        max_val = values.max()

    bins = np.linspace(min_val, max_val, k + 1)
    states = np.digitize(values, bins[1:-1]) # rezultate intre 0 si k-1

    A = np.zeros((k, k))
    for t in range(len(states) - 1):
        i = states[t]
        j = states[t + 1]
        A[i, j] += 1

    row_sums = A.sum(axis=1, keepdims=True)
    A = np.divide(A, row_sums, out=np.zeros_like(A), where=row_sums != 0)
    return A.ravel()

def markov_features(signal, k=6, mins=None, maxs=None):
    feats = []
    for axis in range(3):
        min_val = None if mins is None else mins[axis]
        max_val = None if maxs is None else maxs[axis]
        feats.append(markov_axis_features(signal[:, axis], k, min_val, max_val))
    return np.concatenate(feats)
```

## 9.4 Recomandare pentru intervale

Pentru rezultate stabile, poti calcula min/max pe train pentru fiecare axa si sa folosesti aceleasi intervale pentru train, validation si test.

```
# signals_train este lista de semnale fixate sau brute
all_values = np.concatenate(signals_train, axis=0) # toate momentele din train
mins = all_values.min(axis=0) # min pe x,y,z
maxs = all_values.max(axis=0) # max pe x,y,z

X_train_markov = np.array([markov_features(s, k=6, mins=mins, maxs=maxs) for s in
    signals_train])
X_test_markov = np.array([markov_features(s, k=6, mins=mins, maxs=maxs) for s in
    signals_test])
```

## 9.5 Modele pe Markov features

```
# KNN Manhattan
model = KNeighborsClassifier(n_neighbors=3, metric="manhattan")
model.fit(X_train_markov, y_train)
pred = model.predict(X_val_markov)

# SVM cu kernel intersection
K_train = intersection_kernel_matrix(X_train_markov, X_train_markov)
K_val = intersection_kernel_matrix(X_val_markov, X_train_markov)

model = SVC(C=10, kernel="precomputed")
model.fit(K_train, y_train)
pred = model.predict(K_val)
```

# 10. Perceptron, Widrow-Hoff si MLP

## 10.1 Perceptron

Perceptronul este un clasificator liniar. Calculeaza un scor din produsul scalar intre features si ponderi plus bias.

```
score = np.dot(x, W) + b
y_hat = np.sign(score)
```

Daca datele nu pot fi separate liniar, perceptronul simplu nu poate obtine 100%. XOR este exemplul clasic.

## 10.2 Widrow-Hoff

Widrow-Hoff foloseste loss patrat si actualizeaza ponderile cu gradient descent pentru fiecare exemplu.

```
def train_widrow_hoff(X, y, lr=0.1, epochs=70):
    W = np.zeros(X.shape[1])
    b = 0.0

    for epoch in range(epochs):
        indices = np.random.permutation(len(X))
        for idx in indices:
            x = X[idx]
            target = y[idx]

            y_hat = np.dot(x, W) + b
            error = y_hat - target

            W = W - lr * error * x
            b = b - lr * error

    return W, b
```

## 10.3 MLPClassifier cu sklearn

MLP = retea feed-forward. Datele merg input -> hidden layers -> output. Pentru subiectele practice, varianta sklearn este cea mai rapida.

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(
    hidden_layer_sizes=(128, 64),
    activation="relu",
    solver="adam",
    alpha=0.0001,
    learning_rate_init=0.001,
    max_iter=500,
    random_state=42
)

model.fit(X_train, y_train)
pred = model.predict(X_val)
```

| Parametru                   | Ce inseamna                                        |
|-----------------------------|----------------------------------------------------|
| hidden_layer_sizes=(128,64) | doua straturi ascunse, cu 128 si 64 neuroni        |
| activation='relu'           | functia de activare; alternativa: tanh, logistic   |
| solver='adam'               | optimizator folosit pentru actualizarea ponderilor |
| alpha                       | regularizare L2; reduce overfitting                |
| learning_rate_init          | rata de invatare                                   |
| max_iter                    | numar maxim de epoci/iteratii                      |

## 10.4 PyTorch minim

PyTorch este util daca se cere explicit. Pentru colocviu, trebuie sa recunosti structura: nn.Module, forward, loss, optimizer, backward, step.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28 * 28, 512)
        self.fc2 = nn.Linear(512, 10)

    def forward(self, x):
```

```

        x = self.flatten(x)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = NeuralNetwork()
loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1e-2)

# intr-o bucla de training:
optimizer.zero_grad()
pred = model(image_batch)
loss = loss_function(pred, labels_batch)
loss.backward()
optimizer.step()

```

## 11. Laborator 7 - features de imagine

Laboratorul 7 cere extragere de features din imagini si folosirea lor in modele clasice.

### 11.1 Descriptor tip LBP / histograme de pattern-uri binare

Pentru fiecare pixel, compari intensitatea lui cu vecinii dintr-o fereastră d x d. Rezulta un vector binar. Pentru imagine faci histograma vectorilor unici.

```

def binary_patterns_histogram(image, d=3):
    pad = d // 2
    patterns = {}

    for i in range(pad, image.shape[0] - pad):
        for j in range(pad, image.shape[1] - pad):
            center = image[i, j]
            bits = []
            for di in range(-pad, pad + 1):
                for dj in range(-pad, pad + 1):
                    if di == 0 and dj == 0:
                        continue
                    bits.append(1 if image[i + di, j + dj] >= center else 0)
            pattern = tuple(bits)
            patterns[pattern] = patterns.get(pattern, 0) + 1

    return patterns

```

Pentru modele ML, transformi dictionarele in vectori cu acelasi vocabular de pattern-uri.

### 11.2 Gradient magnitudine

Gradientul masoara schimbarea intensitatii. Magnitudinea combina gradientul pe x si pe y.

```

def gradient_magnitude(image):
    Gx = np.zeros_like(image, dtype=float)
    Gy = np.zeros_like(image, dtype=float)

    Gx[:, 1:-1] = image[:, 2:] - image[:, :-2]
    Gy[1:-1, :] = image[2:, :] - image[:-2, :]

    G = np.sqrt(Gx ** 2 + Gy ** 2)
    return G, Gx, Gy

```

### 11.3 Primele k regiuni 3 x 3 cu magnitudine medie mare

```

def top_k_regions_features(image, k=20, region_size=3):
    G, _, _ = gradient_magnitude(image)
    regions = []

    for i in range(0, image.shape[0] - region_size + 1, region_size):
        for j in range(0, image.shape[1] - region_size + 1, region_size):
            region_G = G[i:i+region_size, j:j+region_size]
            score = region_G.mean()
            regions.append((score, i, j))

    regions = sorted(regions, reverse=True)[:k]

    feats = []

```

```

for score, i, j in regions:
    patch = image[i:i+region_size, j:j+region_size]
    feats.extend(patch.ravel())

return np.array(feats)

```

## 11.4 KNN cu Hamming

```

def hamming_distance(x, y):
    return np.sum(x != y)

def knn_hamming_predict_one(X_train, y_train, x_test, k=3):
    dists = np.array([hamming_distance(x_test, x) for x in X_train])
    idx = np.argsort(dists)[:k]
    return np.bincount(y_train[idx]).argmax()

```

## 11.5 SVM intersection pe histograme

```

K_train = intersection_kernel_matrix(X_train_hist, X_train_hist)
K_val = intersection_kernel_matrix(X_val_hist, X_train_hist)

model = SVC(C=10, kernel="precomputed")
model.fit(K_train, y_train)
pred = model.predict(K_val)

```

# 12. Template-uri finale pentru variante de subiect

## 12.1 Subiect text: string kernel + KNN

```

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

train_texts = np.load("train_data.npy", allow_pickle=True)
y = np.load("train_labels.npy")
test_texts = np.load("test_data.npy", allow_pickle=True)

tr_texts, val_texts, y_tr, y_val = train_test_split(
    train_texts, y, test_size=0.2, random_state=42, stratify=y
)

pred_val = np.array([
    predict_knn_string(tr_texts, y_tr, text, k=3, p=8)
    for text in val_texts
])
print("acc:", accuracy_score(y_val, pred_val))

pred_test = np.array([
    predict_knn_string(train_texts, y, text, k=3, p=8)
    for text in test_texts
])
np.savetxt("subiect2_solutie_knn.txt", pred_test, fmt="%d")

```

## 12.2 Subiect text: string kernel + KRR

```

from sklearn.kernel_ridge import KernelRidge

K_tr = build_kernel_matrix(tr_texts, tr_texts, string_kernel)
K_val = build_kernel_matrix(val_texts, tr_texts, string_kernel)

for alpha in [1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1]:
    model = KernelRidge(alpha=alpha, kernel="precomputed")
    model.fit(K_tr, y_tr)
    scores = model.predict(K_val)
    pred = np rint(scores).astype(int)
    pred = np.clip(pred, y.min(), y.max())
    print(alpha, accuracy_score(y_val, pred))

# dupa ce alegi best_alpha:
K_full = build_kernel_matrix(train_texts, train_texts, string_kernel)
K_test = build_kernel_matrix(test_texts, train_texts, string_kernel)
model = KernelRidge(alpha=best_alpha, kernel="precomputed")
model.fit(K_full, y)
scores = model.predict(K_test)
pred_test = np rint(scores).astype(int)

```

```
pred_test = np.clip(pred_test, y.min(), y.max())
np.savetxt("subiect4_solutie_krr.txt", pred_test, fmt="%d")
```

## 12.3 Subiect accelerometru: citire + Markov + KNN/SVM

```
import pandas as pd

train_df = pd.read_csv("data/train.txt") # coloane: filename,label
test_df = pd.read_csv("data/test.txt") # coloana: filename

signals = []
labels = []
for _, row in train_df.iterrows():
    s = np.loadtxt("data/train/" + row["filename"])
    signals.append(s)
    labels.append(row["label"])
y = np.array(labels)

signals_test = [np.loadtxt("data/test/" + fn) for fn in test_df["filename"]]

all_values = np.concatenate(signals, axis=0)
mins = all_values.min(axis=0)
maxs = all_values.max(axis=0)

X = np.array([markov_features(s, k=6, mins=mins, maxs=maxs) for s in signals])
X_test = np.array([markov_features(s, k=6, mins=mins, maxs=maxs) for s in signals_test])

X_tr, X_val, y_tr, y_val = train_test_split(X, y, test_size=0.2,
                                             random_state=42, stratify=y)

model = KNeighborsClassifier(n_neighbors=3, metric="manhattan")
model.fit(X_tr, y_tr)
pred_val = model.predict(X_val)
print(accuracy_score(y_val, pred_val))
```

## 12.4 MLP pe semnale fixate

```
# alegi target_len, de exemplu lungimea mediana din train
lengths = np.array([s.shape[0] for s in signals])
target_len = int(np.median(lengths))

X = np.array([fix_length(s, target_len).reshape(-1) for s in signals])
X_test = np.array([fix_length(s, target_len).reshape(-1) for s in signals_test])

scaler = StandardScaler()
X_tr, X_val, y_tr, y_val = train_test_split(X, y, test_size=0.2,
                                             random_state=42, stratify=y)
X_tr = scaler.fit_transform(X_tr)
X_val = scaler.transform(X_val)

model = MLPClassifier(hidden_layer_sizes=(128, 64), activation="relu",
                      solver="adam", max_iter=500, random_state=42)
model.fit(X_tr, y_tr)
pred = model.predict(X_val)
print(accuracy_score(y_val, pred))
```

## 13. Greseli frecvente

- La kernel='precomputed' dai X\_train in loc de K\_train. Corect: fit(K\_train, y\_train).
- Calculezi K\_test ca test x test. Corect: test x train.
- Uiti sa faci np.clip dupa KRR si apar clase invalide.
- Folosesti np.argsort crescator la similaritate. Pentru similaritate trebuie np.argsort(-sims).
- Faci fit la StandardScaler pe test. Corect: fit doar pe train, transform pe val/test.
- Ai liste cu lungimi diferite si incerci sa le bagi direct in model. Trebuie padding/truncare/features fixe.
- Confunzi  $x * y$  cu  $\text{np.matmul}(x, y)$ .  $x*y$  este element cu element, matmul este produs matricial.
- Uiti `fmt='%d'` la `np.savetxt` si salvezi 1.0 in loc de 1.
- La Naive Bayes nu folosesti log si ajungi la underflow cu produse de probabilitati mici.

- La KNN cu etichete care nu pornesc de la 0, np.bincount poate fi incomod. Solutie: LabelEncoder sau mapare la 0..C-1.

## 14. Ce inveti prima data, in ordine

Daca timpul este scurt, invata in ordinea asta:

- NumPy: shape, axis, reshape, ravel, load/loadtxt, argsort, bincount, savetxt.
- Train/validation split si accuracy.
- KNN numeric: L1, L2, Manhattan.
- String kernel pe n-grame si KNN cu similaritate.
- Matrice kernel: train x train, val/test x train.
- KRR cu kernel='precomputed', alpha, np rint, np.clip.
- Markov features pentru semnale si KNN/SVM pe ele.
- SVM cu kernel intersectie.
- MLPClassifier simplu cu Adam.
- Naive Bayes si discretizare.
- Features de imagine din laboratorul 7, daca mai ai timp.

## 15. Mini-glosar

| Termen       | Explicatie                                              |
|--------------|---------------------------------------------------------|
| feature      | valoare numerica prin care descrii un exemplu           |
| X            | matricea de exemple: nr_exemple x nr_features           |
| y            | vectorul etichetelor                                    |
| kernel       | functie de similaritate intre doua exemple              |
| K_train      | matricea kernel train x train                           |
| K_test       | matricea kernel test x train                            |
| padding      | adaugi zerouri ca exemplele scurte sa aiba lungime fixa |
| truncare     | tai exemplele lungi la o lungime fixa                   |
| regularizare | penalizare care reduce overfitting-ul                   |
| alpha        | regularizarea la KRR                                    |
| C            | parametru SVM; controleaza penalizarea greselilor       |
| np rint      | rotunjeste scoruri reale la cel mai apropiat intreg     |
| np.clip      | limiteaza valori intre minim si maxim                   |

## Surse folosite

Documentul sintetizeaza materialele si modelele de colocviu discutate: Laboratoarele 1-7, Subiect-2021, varianta cu texte si KRR, si varianta cu semnale accelerometru, Markov features, KNN, SVM si MLP. Codurile sunt template-uri didactice si trebuie adaptate la numele exacte ale fisierelor din enunt.